



ARM7TDMI[®]
DesignStart[™] Kit
User Guide

DesignStart Program

ARM7TDMI DesignStart Kit User Guide

© Copyright ARM Limited 2005. All rights reserved.

Proprietary notice

Words and logos marked with ® or ™ are registered trademarks or trademarks owned by ARM Limited, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Confidential. This information can only be distributed to ARM employees, employees of companies that have signed a Non-Disclosure Agreement (NDA) with ARM, and employees of companies that have signed an NDA with them.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the product

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Change history

Issue	Date	Comments
8.0	7 April 2005	First release.

Contents

1	ABOUT THIS DOCUMENT	6
1.1	Scope	6
1.2	References	6
1.3	Terms and abbreviations	6
2	INTRODUCTION	8
2.1	About the ARM7TDMI DesignStart Kit	8
2.2	DesignStart Kit Key Components	8
2.2.1	ARM7TDMI Foundry Specific Kit	8
2.2.2	ARM7TDMI Integration Kit	8
2.2.3	ARM7TDMI Design Simulation Model (DSM)	9
2.3	Differences Between the DesignStart Kit and Full Single Use Design License (SUDL) Kit	9
2.4	Obtaining Support for the DesignStart Kit	9
2.5	EDA Tool Versions	10
3	DESIGNSTART KIT OVERVIEW	11
3.1	Directory Structure	11
3.2	Getting Started	12
3.2.1	Preparatory Steps	12
3.3	Example Environments	12
3.3.1	AMBA Environment	12
3.3.2	JTAG Environment	13
4	AMBA ENVIRONMENT SIMULATION AND SYNTHESIS	14
4.1	Introduction	14
4.1.1	Background Information	14
4.2	Simulation of the AMBA Environment	14
4.2.1	Environment Overview	14
4.2.2	Simulating the Design	14
4.2.3	Analyzing the Results	15
4.3	Synthesis of the AHB Wrapper	16
4.3.1	Synthesis Overview	16
4.3.2	Configuration	16

4.3.3	Synthesizing the Design	16
4.3.4	Analyzing the Results	16
5	JTAG ENVIRONMENT SIMULATION AND SYNTHESIS	18
5.1	Introduction	18
5.1.1	Background Information	18
5.2	Simulation of the JTAG Environment	18
5.2.1	Environment Overview	18
5.2.2	Simulating the Design	18
5.2.3	Analyzing the Results	19
6	FURTHER PROGRESS	20

1 ABOUT THIS DOCUMENT

1.1 Scope

This document describes the structure and use of the ARM7TDMI DesignStart Kit for both AMBA and JTAG environments. In addition to introducing the kit, it covers the directory structure, running manufacturing test vectors and performing synthesis of the AMBA AHB wrapper.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
[1]	Foundry Specific	ARM7TDMI Core Configuration and Performance Summary
[2]	ARM DII 0101	ARM7TDMI Revision: Rev3a and r3p0 Integration Manual
[3]	ARM DDI 0029	ARM7TDMI Rev3 Technical Reference Manual
[4]	ARM DDI 0169	AHB Wrappers Technical Reference Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations:

Term	Meaning
AHB	Advanced High-Performance Bus. A particular interface defined in the AMBA specification commonly found on later ARM processors. For ARM processors that do not have an AHB interface as standard, a component called an AHB wrapper is available to convert the processor's interface to be AHB compliant.
AMBA [®]	Advanced Microcontroller Bus Architecture. A standard promoted by ARM that permits simplified SoC design by defining the interface of IP components and the way they are connected together. The AMBA specification can be downloaded from the ARM website.
ARM7TDMI	ARM7TDMI processor core. Most popular 32-Bit RISC Architecture CPU core available on a large number of foundry processes.
DFT	Design For Test. A term applied to all aspects of a design concerning how the design will be tested after manufacture to ensure it exhibits no manufacturing defects.
DSM	Design Simulation Model. An abstracted cycle accurate model of an ARM CPU core used for the purposes of functional simulation.

Term	Meaning
IM	Integration Manual. A document describing how to integrate an ARM IP component into a typical design flow.
RTL	Register Transfer Level. A description of a design in a high level text language (e.g. Verilog) that supports direct synthesis to a target Silicon process.
SoC	System On Chip.
SUDL	Single Use Design License. A license provided to ARM permitting manufacturing of an ARM7TDMI based design on a given foundry process.
Testbench	A component used at the point of verification of a design to simulate an SoC design.
TIC	Test Interface Controller. An on-chip RTL block used to generate AMBA transactions based on test vectors applied by the Ticbox through the test bus.
Ticbox	A testbench component used to pass test vectors to the a TIC controller present in the SoC.
TRM	Technical Reference Manual. A processor specific document that describes the interfaces and the behaviour of the particular ARM processor.

2 INTRODUCTION

2.1 About the ARM7TDMI DesignStart Kit

The ARM7TDMI DesignStart kit contains key components of the ARM7TDMI design environment to enable an SoC designer to incorporate an ARM7TDMI processor core into their SoC implementation. All key front end views of the processor core (functional, timing, test) plus backend physical views (Synopsys and Cadence flows) are included in the kit.

In addition to providing the key abstract views of the core, some ARM specific additional AMBA infrastructure IP is also included, plus example simulation and synthesis environments.

2.2 DesignStart Kit Key Components

The DesignStart kit comprises three key sets of components delivered as individual tar files. These components and their constituent parts are described below.

2.2.1 ARM7TDMI Foundry Specific Kit

These components are specific to your given choice of foundry process. Foundry specific data supplied is as follows:

- ARM7TDMI Core Configuration and Performance Summary [Ref 1] – this document gives process specific information on the core for your chosen foundry process.
- .lib Timing Views – abstracted foundry specific timing views of the core in industry standard Liberty .lib format for a number of process conditions. Pre-compiled Synopsys db views are also supplied.
- Synopsys Physical Abstract and Timing Views: FRAM, CLF.
- Cadence Physical Abstract Views: LEF.

2.2.2 ARM7TDMI Integration Kit

These components are common across all foundry processes. Integration data supplied is as follows:

- ARM7TDMI DesignStart Kit User Guide (this document) – this guide describes the deliverables shipped with the kit and how to use the example environments provided.
- ARM7TDMI Integration Manual (IM) [Ref 2] – this is a comprehensive document that describes how to integrate the ARM7TDMI into a typical SoC design flow.
- ARM7TDMI Technical Reference Manual (TRM) [Ref 3] – this document describes the internal and external interface operation of the ARM7TDMI core.
- AHB Wrappers Technical Reference Manual (TRM) [Ref 4] – this document describes the AMBA AHB Wrapper for the ARM7TDMI core.
- AMBA AHB Infrastructure and example environment – AMBA AHB Wrapper synthesizable Verilog, plus synthesizable AMBA Test Infrastructure (for testing the ARM7TDMI core). Example synthesis scripts are also provided to synthesize the AHB wrapper using Synopsys Design Compiler.
- Example test vectors– example test vectors for testing the core using the ARM7TDMI JTAG port or using the AMBA TIC methodology.
- Example Vector Replay Environments – example environments for replaying the supplied test vectors on an SoC design for JTAG and AMBA TIC test methodologies.

2.2.3 ARM7TDMI Design Simulation Model (DSM)

This component provides an abstracted simulation model for the ARM7TDMI processor core. The model supplied is platform (Solaris, or Linux) and Verilog simulator (Cadence NC-Sim, Synopsys VCS, or Mentor ModelSim) specific.

It is very important that you have the correct DSM for your Verilog simulator and platform.

2.3 Differences Between the DesignStart Kit and Full Single Use Design License (SUDL) Kit

The DesignStart kit is provided to allow SoC designers to integrate the ARM7TDMI into their SoC design and progress through physical implementation. However, no manufacturing rights are included in this kit. **In order to manufacture your design at your chosen foundry you require a Single Use Design License (SUDL) from ARM.**

In obtaining a SUDL license you gain the following benefits:

- Rights to manufacture your design at your chosen foundry.
- Direct technical support from ARM - no direct engineering support from ARM is provided with the DesignStart kit.
- Full feature DSM Model – a DSM model permitting instruction and register tracing and the ability to run for an unlimited number of cycles. **The DesignStart kit DSM limits the total cycle count (TCK or MCLK) to 5 million cycles. The DesignStart kit DSM does not allow register or instruction tracing.**
- AMBA Design Kit (ADK) – the AMBA Design Kit offers a variety of pre-configured AMBA AHB systems (including a full working microcontroller design) plus all AMBA AHB infrastructure components. The DesignStart Kit only includes the ARM proprietary AHB components used for testing the ARM7TDMI core.
- Full test vectors – a full set of test vectors for manufacturing testing the ARM7TDMI core. **The DesignStart kit contains a limited subset of test vectors.**
- Antenna Views and GDSII footprint – final physical views for signoff and tapeout to foundry. **The DesignStart kit does not contain the antenna and GDSII footprint views required in the final signoff flow.**
- Cadence Power and Noise Views – views for Cadence tools to support power and noise analysis. The DesignStart kit does not contain these views.
- Software and System Development tools, such as software development tools and system development hardware.

2.4 Obtaining Support for the DesignStart Kit

No direct engineering support is provided by ARM for the DesignStart kit. However, the following resources are available:

- Technical documentation supplied with the DesignStart kit
- Applications notes available on www.arm.com
- FAQ entries relating to this kit and the ARM7TDMI, available in the download area of www.artisan.com
- Submission to on-line forum, available in the download area of www.artisan.com

2.5 EDA Tool Versions

The Integration kit has been tested using the EDA tools shown in Table 1. The foundry specific timing and physical views have been verified with a range of Synopsys and Cadence tool versions.

In addition, the supplied industry standard views are compatible with tools from other EDA vendors. Please consult your local EDA vendor representative for details on converting these views for use with other EDA tools. ARM does not accept liability for translated library formats.

Table 1 - Tool Versions

Task	Tool Vendor / Version
Simulation	Mentor ModelSim 6.0b Cadence LDV 5.1 Synopsys VCS 7.2
Synthesis	Synopsys Design Compiler 2004.06-SP1-1

3 DESIGNSTART KIT OVERVIEW

This chapter provides an overview of DesignStart kit directory structure plus an introduction to the example environments supplied.

3.1 Directory Structure

Figure 1 shows the supplied directory structure of each of the key components of the DesignStart kit. It is important that you ensure the directory structure in Figure 1 (3) is preserved by copying the components of (1) and (2) together to form the required structure- this will ensure that steps in the following chapters proceed smoothly.

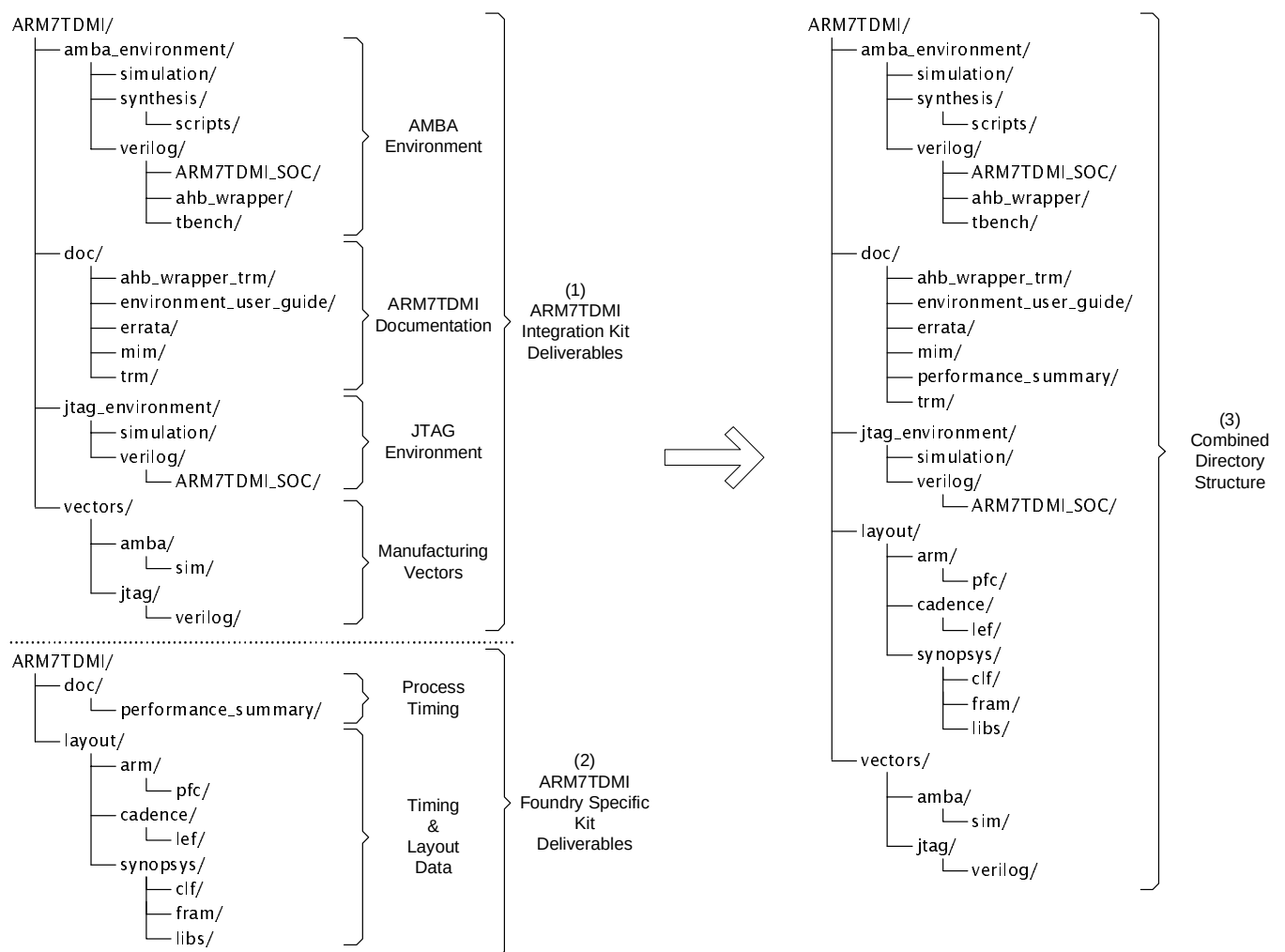


Figure 1 - Directory Structure

3.2 Getting Started

It is recommended that you unpack the supplied tar file deliverables and familiarise yourself with the deliverables, following the examples in the remaining sections in this guide in conjunction with the Integration Manual [Ref 2]. Moving forward, the ARM7TDMI Integration Manual [Ref 2] and ARM7TDMI Technical Reference Manual [Ref 3] provide you with detail on building the ARM7TDMI into your own SoC design. You can use the example systems provided in the DesignStart kit to understand how to support the ARM7TDMI test methodology in your design.

3.2.1 Preparatory Steps

The following steps are required before proceeding with the following chapters:

1. Unpack the Integration Kit and Foundry Specific kit tar files provided by ARM, recreating the directory structure shown in Figure 1 (3).
2. Unpack and install the DSM model (ensuring you have the correct DSM for your chosen Verilog simulator and platform). The DSM install script assumes the presence of 'gtar'- if you do not have this you may have to modify the install script to use 'tar'. The file 'install.notes' in the DSM tar file provides details on installing the DSM- it can be installed at your chosen location (it does not have to be within the main deliverables directory tree). You must ensure that the DSM is installed correctly (by running the crftester supplied with the DSM) before proceeding. **After installing the DSM, you are advised to make a note of the environment variables in Table 2 as these variables must be set each time you run a simulation.**

Table 2 - DSM Environment Variables

Environment Variable	Setting
MG_LIB	
ARM7TDMI_HOME	
DIR_ARM7TDMI	
LD_LIBRARY_PATH (Synopsys VCS Only)	

3.3 Example Environments

To aid understanding of the ARM7TDMI and its test environment, two simple example systems are provided with the DesignStart kit. These example systems are provided to demonstrate how the ARM7TDMI can be manufacturing tested based on the given integration strategy of the core (AMBA wrapped or native interface). Additionally, the kit demonstrates how the AMBA AHB wrapper is integrated with the core and how it is tested.

The remaining chapters in this user guide discuss the example environments, how they are simulated and, where applicable, synthesized.

3.3.1 AMBA Environment

The AMBA environment (shown in Figure 2) is a simple AMBA AHB SoC, containing an ARM7TDMI core surrounded by an AMBA AHB wrapper, a low gate-count Test Interface Controller (TIC), and some additional control logic. The TIC connections are brought up to the top-level. This example demonstrates the TIC controller and AHB wrapper connectivity, and can be used as the basis for developing an AMBA based system.

In this example system, the TIC acts as an AHB master, and the ARM7TDMI wrapper acts as an AHB slave, with a direct connection between the two blocks. The AHB master port of the ARM7TDMI wrapper is not used. Note that this is a basic example- a typical AMBA AHB SoC would contain other peripherals accessed by the ARM7TDMI as well as AMBA bus infrastructure blocks (arbiter, decoder, multiplexers).

The TIC test methodology makes use of existing off-chip bidirectional signals (such as a memory data bus) along with three test control signals to apply 32-bit parallel test vectors to the ARM7TDMI core, and any other suitable AMBA components.

An example simulation environment is supplied that applies external test vectors through the TIC. Additionally, synthesis scripts are provided as an example of how the ARM7TDMI AMBA AHB wrapper can be synthesized.

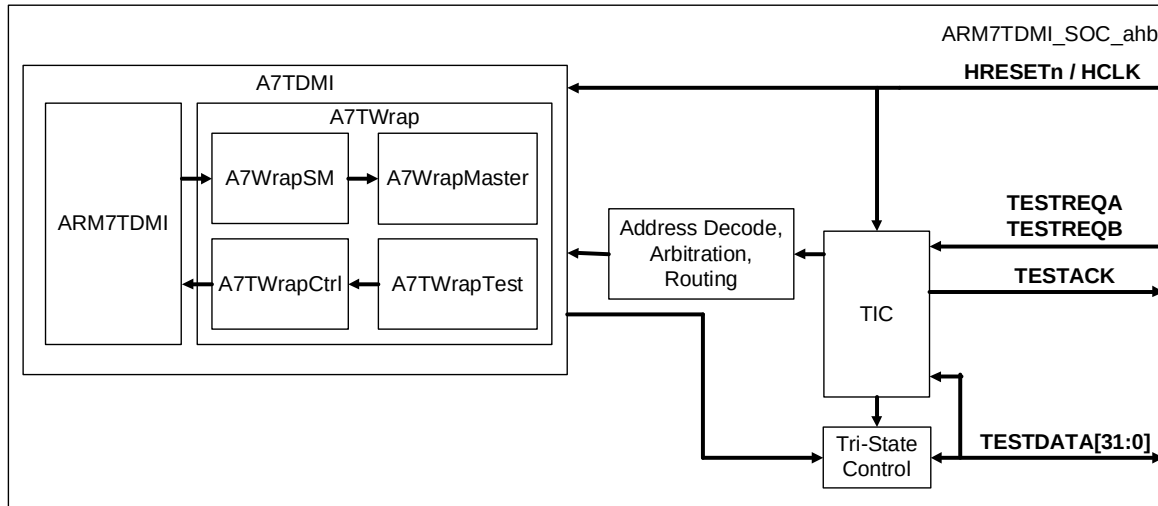


Figure 2 - AMBA Example SoC Overview

3.3.2 JTAG Environment

The JTAG environment (shown in Figure 3) includes an example of a simple SoC, containing an ARM7TDMI core with the JTAG pins brought up to the top-level. This can be used as the basis for developing a system using the native ARM7TDMI interface where the JTAG test methodology is used to apply serial test vectors to the ARM7TDMI core. An example simulation environment is supplied that applies external test vectors through the JTAG interface.

A typical SoC would contain other components such as peripherals accessed by the ARM7TDMI and system infrastructure blocks.

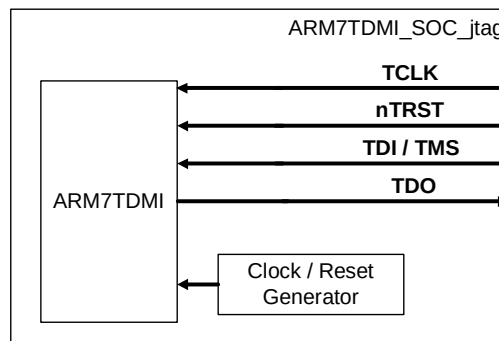


Figure 3 - JTAG Example SoC Overview

4 AMBA ENVIRONMENT SIMULATION AND SYNTHESIS

4.1 Introduction

This chapter provides detail on the example simulation and synthesis environment supplied for an AHB wrapped ARM7TDMI core. This environment forms the basis of a design using the AMBA AHB standard.

4.1.1 Background Information

Useful background information to this chapter is provided in the following documents:

- DFT Guidelines Chapter of the ARM7TDMI Integration Manual [Ref 2]
- DFT Verification Chapter of the ARM7TDMI Integration Manual [Ref 2]
- AMBA AHB Wrappers Technical Reference Manual [Ref 4]

4.2 Simulation of the AMBA Environment

4.2.1 Environment Overview

An overview of the AMBA simulation environment is given in Figure 4. This environment shows:

- The example AMBA SoC (ARM7TDMI_SOC_ahb)
- Testbench (tbench) that applies test vectors to the core through the behavioral Ticbox model
- Vector file 'infile.sim' compiled with the Ticbox

This environment demonstrates:

- Integration of the AMBA AHB wrapper around the ARM7TDMI processor
- Basic connectivity of TIC controller with AHB wrapper
- Replay of manufacturing test patterns on the core using the wrapper

4.2.2 Simulating the Design

Before simulating the design, it is assumed that you have installed the DSM correctly and the environment variables defined in Table 2 have been set.

All steps to simulate the design are performed in the directory `amba_environment/simulation`. The following steps are required to run a simulation of an AMBA vector:

1. Create a link to the AMBA vector to use for the simulation with the following command:

```
$prompt> ln -f -s ../../vectors/amba/sim/<vector_name>.sim infile.sim
```
2. Run the simulator (with an optional compile step if required) on the Verilog files listed in `files_ahb.vc`, using `tbench` as the top-level block.

Note: the file 'infile.sim' is included in the Ticbox model using a ``include` Verilog directive. This means that each time you wish to use a different vector, you must recompile the file 'Ticbox.v'.

Included in the `amba_environment/simulation` directory are three UNIX tc-shell scripts that compile and run the AMBA verification vectors for your chosen verilog simulator:

```
run_amba_tests_modelsim.tcsh      : To use with Mentor ModelSim simulator
run_amba_tests_ncverilog.tcsh     : To use with Cadence NC Verilog simulator
run_amba_tests_vcs.tcsh          : To use with Synopsys VCS simulator
```

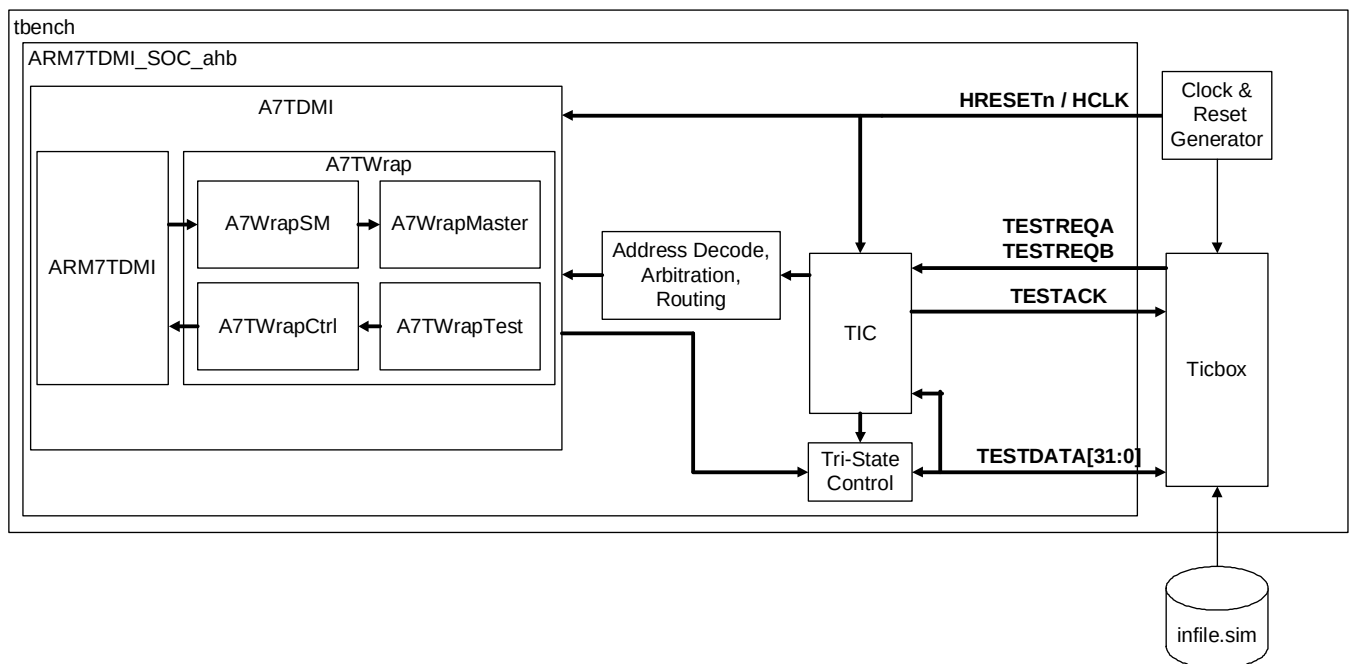


Figure 4 - AMBA Simulation Environment

4.2.3 Analyzing the Results

The DSM is expected to generate error messages during initialisation at time 0, you may also see some warnings reported by the DSM starting “ **** Warning: Illegal** ”- further detail on these messages is provided in Section 6.1.1 of the Integration Manual. The test has passed if no additional errors are reported and the `Test sequence completed` message is displayed at the end of the simulation.

Three directories with reference simulation logs to allow result comparisons with the actual run are also supplied:

```
ref_logs_modelsim      : Mentor ModelSim reference simulation logs
ref_logs_ncverilog     : Cadence NC Verilog reference simulation logs
ref_logs_vcs           : Synopsys VCS reference simulation logs
```

4.3 Synthesis of the AHB Wrapper

4.3.1 Synthesis Overview

The supplied synthesis environment demonstrates an example of how the ARM7TDMI AHB wrapper can be synthesized- it is provided as an example only and typically you will replace these scripts with your own top level implementation script. The synthesis scripts supplied are in Synopsys TCL format. The constraints and flow used in the supplied scripts can be used to generate scripts compatible with a different synthesis tool.

Synthesis of the ARM7TDMI AHB wrapper is performed in the `amba_environment/synthesis` directory using a makefile which calls the top-level TCL synthesis script `scripts/A7TWrap_compile.tcl`.

The synthesis scripts must be modified before use to set the target clock frequency and all variables for the target cell library, details are given below.

4.3.2 Configuration

Before running synthesis, you should ensure that the directory structure described in Figure 1 (3) has been generated.

The following files require to be configured to your local environment prior to running synthesis:

- Modify `scripts/process.tcl` to specify your target process (required modifications are flagged with '## MODIFY ##').
- Modify `scripts/setup.tcl` to specify your location of your library files (required modifications are flagged with '## MODIFY ##').
- If required, change the default target clock period setting of 20.0ns (50MHz) at the start of `scripts/A7TWrap_compile.tcl`

4.3.3 Synthesizing the Design

Synthesis can be run to produce either a logical design (no scan) or a scan inserted design. Use the command "make noscan" to perform synthesis without using scan cells or inserting a scan chain, or "make scaninsert" to perform synthesis using scan cells and inserting a scan chain.

4.3.4 Analyzing the Results

Once synthesis has completed, a number of new directories will have been created (`db`, `logs`, `netlist`, `reports`, `synopsys_cache`, `work`). These directories contain the following:

- `db` - Synopsys db format views of the AHB wrapper top-level and AHB wrapper functional block.
- `logs` - Logs of the synthesis, `*_run.log` showing the output of `dc_shell_t`, and `*_synth.log` showing the output of multiple synthesis check commands.
- `netlist` - Verilog netlist and SDF timing views of the AHB wrapper top-level and AHB wrapper functional block.
- `reports` - Timing and functional reports generated during synthesis. Further detail on these reports is given in Table 3.

Reference logs and reference reports are provided in the directories 'ref_logs' and 'ref_reports' respectively.

Table 3 - Synthesis Report Files

Report name	Description
area	The area for all modules in the design.
cell	A list of all the cells in the design.
constr	General information about the constraints on the netlist that has been created.
hier	The hierarchy of all blocks and cells in the design.
latch	Shows all latches and timing loops in the design. One latch is expected - uA7TWrap/DelCLKEN/iDATAOUT_reg.
max	Maximum timing report using the maximum cell and ARM7TDMI timing libraries.
min	Minimum timing report using the minimum cell and ARM7TDMI timing libraries.
ports	Constraints on the ports.
PreviewAll	Output from the preview scan command.
PreviewScript	The set_scan_path command to define the scan path.
ref	A list of the cell library references used in the design.
scan	Information about the estimated fault coverage and the scan chain connections.
stil	The top-level scan chain connections defined in STIL format.
transitive_fanout	A list of all cells and pins connected to a clock or reset tree.
vio	A summary of all constraint violations. The only acceptable violations are a design area violation or a hold timing violation to the ARM7TDMI block, as the maximum timing library is used to generate the violation reports. The minimum timing report contains any valid hold violations.

5 JTAG ENVIRONMENT SIMULATION AND SYNTHESIS

5.1 Introduction

This chapter provides detail on the simulation environment supplied for a native ARM7TDMI core. This environment forms the basis of a design using the native interface of the ARM7TDMI.

5.1.1 Background Information

Useful background information this chapter is provided in the following documents:

- DFT Guidelines Chapter of the ARM7TDMI Integration Manual [Ref 2]
- DFT Verification Chapter of the ARM7TDMI Integration Manual [Ref 2]
- ARM7TDMI Technical Reference Manual [Ref 3]

5.2 Simulation of the JTAG Environment

5.2.1 Environment Overview

An overview of the JTAG vector simulation environment is given in Figure 5. This environment shows:

- The example native interface SoC (ARM7TDMI_SOC_jtag)
- Embedded Verilog Vector File '<vectorname>.vec' containing a Verilog module 'tbench'. This is a Verilog testbench file that simulates the JTAG inputs and checks the outputs.

This environment demonstrates:

- Replay of manufacturing test patterns on the core using the JTAG interface

5.2.2 Simulating the Design

Simulation of the design is performed in the directory `jtag_environment/simulation`. The following step is required to run a simulation of a JTAG vector:

1. Run the simulator (with an optional compile step if required) on the Verilog files listed in `files_jtag.vc` and the selected test vector `vectors/jtag/verilog/<vector_name>.vec`, using `tbench` as the top-level block.

Included in the `jtag_environment/simulation` directory are three UNIX tc-shell scripts that compile and run the JTAG verification vectors for your chosen verilog simulator:

<code>run_jtag_tests_modelsim.tcsh</code>	: To use with Mentor ModelSim simulator
<code>run_jtag_tests_ncverilog.tcsh</code>	: To use with Cadence NC Verilog simulator
<code>run_jtag_tests_vcs.tcsh</code>	: To use with Synopsys VCS simulator

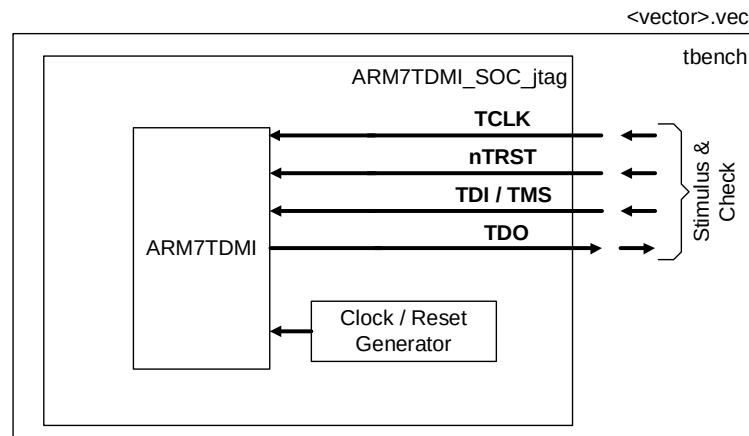


Figure 5 - JTAG Simulation Environment

5.2.3 Analyzing the Results

The DSM is expected to generate error messages during initialisation at time 0, you may also see some warnings reported by the DSM starting “** Warning: Illegal ”- further detail on these messages is provided in Section 6.1.1 of the Integration Manual. The test has passed if no additional errors are reported and the `TEST COMPLETED WITH NO ERRORS` message is displayed at the end of the simulation.

Three directories with reference simulation logs to allow result comparisons with the actual run are also supplied:

<code>ref_logs_modelsim</code>	: Mentor ModelSim reference simulation logs
<code>ref_logs_ncverilog</code>	: Cadence NC Verilog reference simulation logs
<code>ref_logs_vcs</code>	: Synopsys VCS reference simulation logs

6 FURTHER PROGRESS

Detail provided in the previous chapters has demonstrated the ARM specific parts of the SoC design flow.

To proceed further, the ARM7TDMI Integration Manual [Ref 2] provides detail on all aspects of integrating the ARM7TDMI into an SoC design. The Integration Manual also provides suggestions for further reading.

In addition, training courses are available from ARM and other third parties.